

基于 TCP 通信的联邦学习系统

计算机网络大作业 Poster 辅助说明

小组汇报

2026 年 6 月

项目目标：用纯 TCP 跑通联邦学习闭环

- 基础任务：1 个 Server + 3 个 Client 的同步式联邦学习。
- 网络要求：只使用 Python 原生 socket，不使用 HTTP/RPC 传模型。
- 关键难点：模型权重是大 payload，必须解决 TCP 字节流粘包。
- 扩展实现：Ring 去中心化、SplitFed、弱网与掉队者、实时监控。

最终结果

Centralized: 99.00%

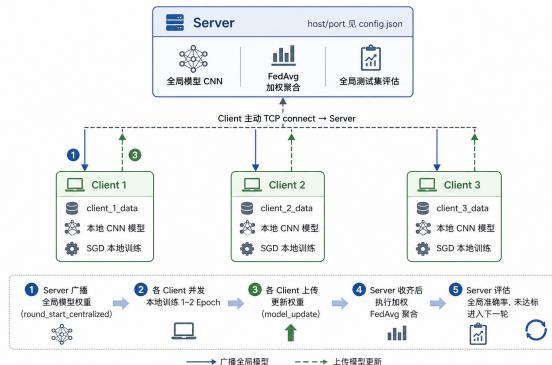
Ring: 99.01%

SplitFed: 99.04%

展示重点

网络协议和弱网容错是本项目的核心，不只是模型训练。

集中式联邦学习架构



- Server 保存全局 CNN 和测试集。
- Client 只读取自己的本地数据文件。
- 每轮流程：广播权重、Client 本地训练、上传更新、FedAvg 聚合。
- 达到目标准确率后广播 shutdown。

自定义 TCP 封包解决粘包问题

Length Header
8 字节

Magic Number
SF26

Payload
pickle + 可选 gzip

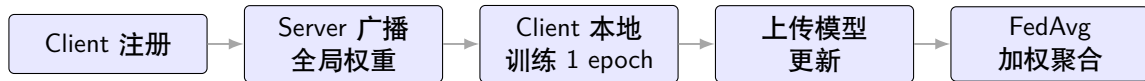
发送端

- `pickle.dumps(message)`
- `struct.pack(">Q", length)`
- `sendall(header + magic + payload)`

接收端

- 先精确读 8 字节长度。
- 再精确读 4 字节魔数。
- 最后循环接收，直到读满 L 字节。

基础训练流程



- 每个 Client 的上传消息包含权重、样本数、训练/验证/测试指标。
- Server 按样本数加权平均: $w = \sum_i \frac{n_i}{\sum_j n_j} w_i$ 。
- 每轮结束后 Server 在全局测试集上评估并记录网络传输统计。

弱网处理：延迟发生在 Client，决策发生在 Server

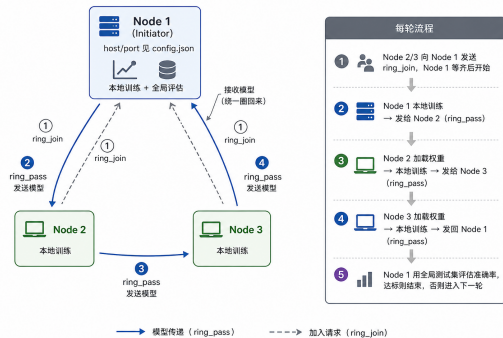
Client 侧模拟

- 配置：client_2.delay = 5.0
- 本地训练完成后，发送更新前 sleep。
- 可配置 drop_rate 直接跳过某轮上传。
- 上报 straggler_delay 监控事件。

Server 侧容错

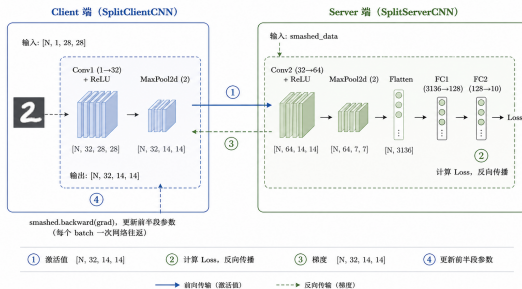
- server_timeout 控制最多等待时间。
- 超时后未返回节点记为 straggler_dropped。
- 若收到数 $\geq \text{min_clients}$ ，继续聚合。
- 否则中止本次训练。

扩展一：Ring 去中心化拓扑



- 废弃中心 Server，三个节点组成环。
- Node 1 训练后发给 Node 2，Node 2 再发给 Node 3。
- 模型绕环返回 Node 1 后进行全局测试。
- 优点是无中心节点；代价是节点之间串行依赖更强。

扩展二：SplitFed 分割联邦



- Client 保存 CNN 前半段，Server 保存后半段。
- Client 每个 batch 发送激活值和标签。
- Server 计算 loss 与梯度，再把激活梯度传回 Client。
- 收敛轮次少，但 batch 级通信导致通信量最大。

三种模式结果对比

模式	完成轮次	测试准确率	运行时长	网络收发量
Centralized	6 / 10	0.9900	192.5s	58.0 MB / 58.0 MB
Ring	9 / 10	0.9901	214.1s	43.5 MB / 43.5 MB
SplitFed	4 / 10	0.9904	334.2s	9.0 GB / 9.0 GB

结论

Centralized 最符合基础要求且总耗时最短；Ring 证明去中心化可行；SplitFed 准确率略高但通信开销最大。

监控面板与训练曲线

